Project Abstract

Project Title

Software Engineering Knowledge & Code Intelligence Platform

Real-World Problem

As software systems grow, they accumulate large numbers of interconnected components — developers, repositories, services, APIs, dependencies, bugs, incidents, and releases. Understanding how these parts relate becomes increasingly difficult. Developers often struggle to identify what may be affected when a component is changed, to locate relevant historical bugs or incidents, or to understand an unfamiliar system when onboarding.

Motivation

Modern AI coding agents (e.g., Claude Code, GitHub Copilot) can read and explain code within a session, but they are stateless — they re-analyze code from scratch each time and have no persistent memory of relationships across repositories, past incidents, or releases. At the scale of a large organization with thousands of files across hundreds of repositories, this makes real-time dependency and impact analysis impractical using code-reading agents alone. This gap — the need for persistent, structured, queryable memory of a software system's relationships — motivated this project.

Proposed Solution

A software engineering intelligence platform that models the structural and historical relationships within a software system in a relational database, supported by a vector database for semantic search over unstructured technical text such as bug and incident descriptions. The system allows developers to query dependency chains, trace potential change impact, and retrieve conceptually related historical issues even when described using different terminology.

Objectives

- Design a relational schema capturing entities such as developers, repositories, services, APIs, dependencies, bugs, incidents, and releases, along with their relationships.
- Support change-impact queries (e.g., identifying downstream services affected by a change to a given API), including multi-level / recursive dependency traversal.
- Integrate a vector database to enable semantic search over bug and incident reports.
- Expose the system through a Next.js web application with authentication and role-based access.
- Demonstrate advanced database concepts including recursive queries, views, triggers, and indexing.

Scope

The project covers schema design, backend API integration, and a functional Next.js interface for querying dependencies, change impact, and historical bug/incident search. The dataset combines real repository structure (sourced via the GitHub API) with simulated bug and incident records, since a fully live deployment integrating GitHub, Jira, and incident-tracking systems in real time is outside the scope of this course project. Static code analysis (auto-detecting dependencies from source) is also outside scope; dependency relationships are curated based on real repository structure.

Functional Requirements

- Users can register/log in with role-based access (e.g., Admin, Developer).
- Users can view and query a service/API dependency graph, including multi-hop change-impact queries.

- Users can search past bugs and incidents using keyword (SQL) and semantic (vector) search.
- Users can view a consolidated onboarding summary for a given service (dependencies, contributors, incident history).
- Admins can manage (add/update/remove) entities such as services, APIs, and dependency records.

Database Choice and Justification

This project follows Option 2 (SQL + Vector Database) as defined in the course requirements. PostgreSQL is used as the mandatory SQL database because the core data — developers, repositories, services, APIs, dependencies, releases — is highly structured and relationship-heavy, requiring primary/foreign keys, constraints, joins, and recursive queries for dependency traversal. A vector database (PostgreSQL with pgvector) is used specifically for bug and incident descriptions, which are unstructured, free-text, and benefit from semantic similarity search rather than exact keyword matching — for example, matching a query about “payment failure after session expiry” to a past incident logged as “token timeout causing payment rejection.” Each database paradigm is therefore chosen to match the nature of the data it stores, not for technology demonstration alone.

Main Features / Functionalities

- Dependency and change-impact visualization for services and APIs.
- Semantic search over historical bugs and incidents.
- Developer and repository contribution history.
- Onboarding view summarizing a service's dependencies, ownership, and incident history.
- Role-based authentication and access control (minimum 2 roles: Admin, Developer).

Expected Outcome / Benefit

A working prototype demonstrating how a hybrid relational and vector database architecture can provide persistent, queryable understanding of a software system's structure and history — a capability that complements (rather than duplicates) code-reading AI agents by supplying the structured, cross-session memory they lack. The project will use a hybrid dataset: real repository structure sourced via the GitHub API combined with simulated bug/incident data, since production-scale deployment would require live integration with tools such as GitHub and incident-tracking systems.